
Lecture 20: Unsupervised Learning

Md. Shahriar Hussain
ECE Department, NSU



Why use Unsupervised Learning?

- Although most of the applications of Machine Learning today are based on supervised learning, the vast majority of the available data is unlabeled
- Supervised Learning relies on large, labeled datasets to work!
- Data collection and labelling is:
 - Expensive
 - Slow
- Supervision → Learn to group objects into one class because someone tells us to
- Experience (Unsupervised) → Learn to group objects into one class by seeing many of them

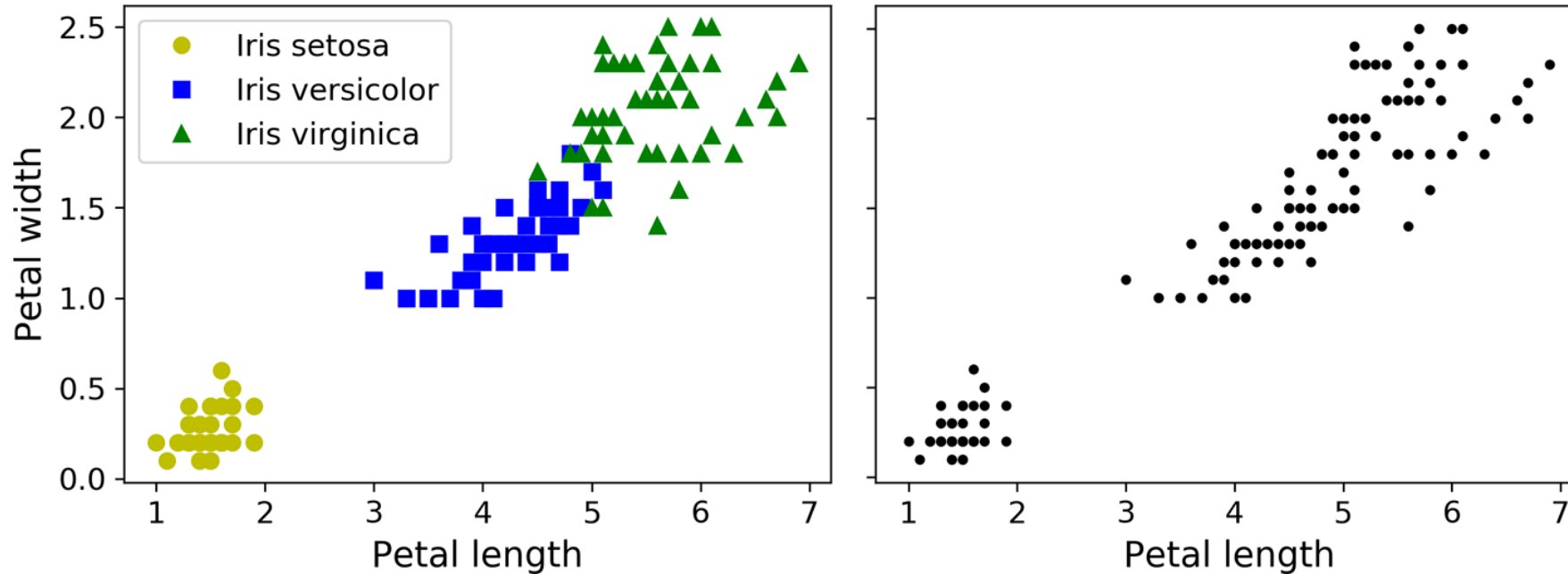


Unsupervised Learning Tasks and Algorithms

- Clustering
 - The goal is to group similar instances together into *clusters*.
 - *Clustering is a great* tool for data analysis, customer segmentation, recommender systems, search engines, image segmentation, semi-supervised learning, dimensionality reduction, and more
- *Anomaly detection*
 - The objective is to learn what “normal” data looks like, and then use that to detect abnormal instances,
 - such as defective items on a production line or a new trend in a time series.



Classification vs Clustering



- On the left is the iris dataset , where each instance's species is represented with a different marker.
- It is a labeled dataset, for which classification algorithms such as Logistic Regression, SVMs, or Random Forest classifiers are well suited.

Classification vs Clustering

- Unlike classification, clustering is an unsupervised task
- On the right is the same dataset, but without the labels, so you cannot use a classification algorithm anymore.



K-Means

- The K-Means algorithm is a simple algorithm capable of clustering unlabeled dataset very quickly and efficiently, often in just a few iterations.

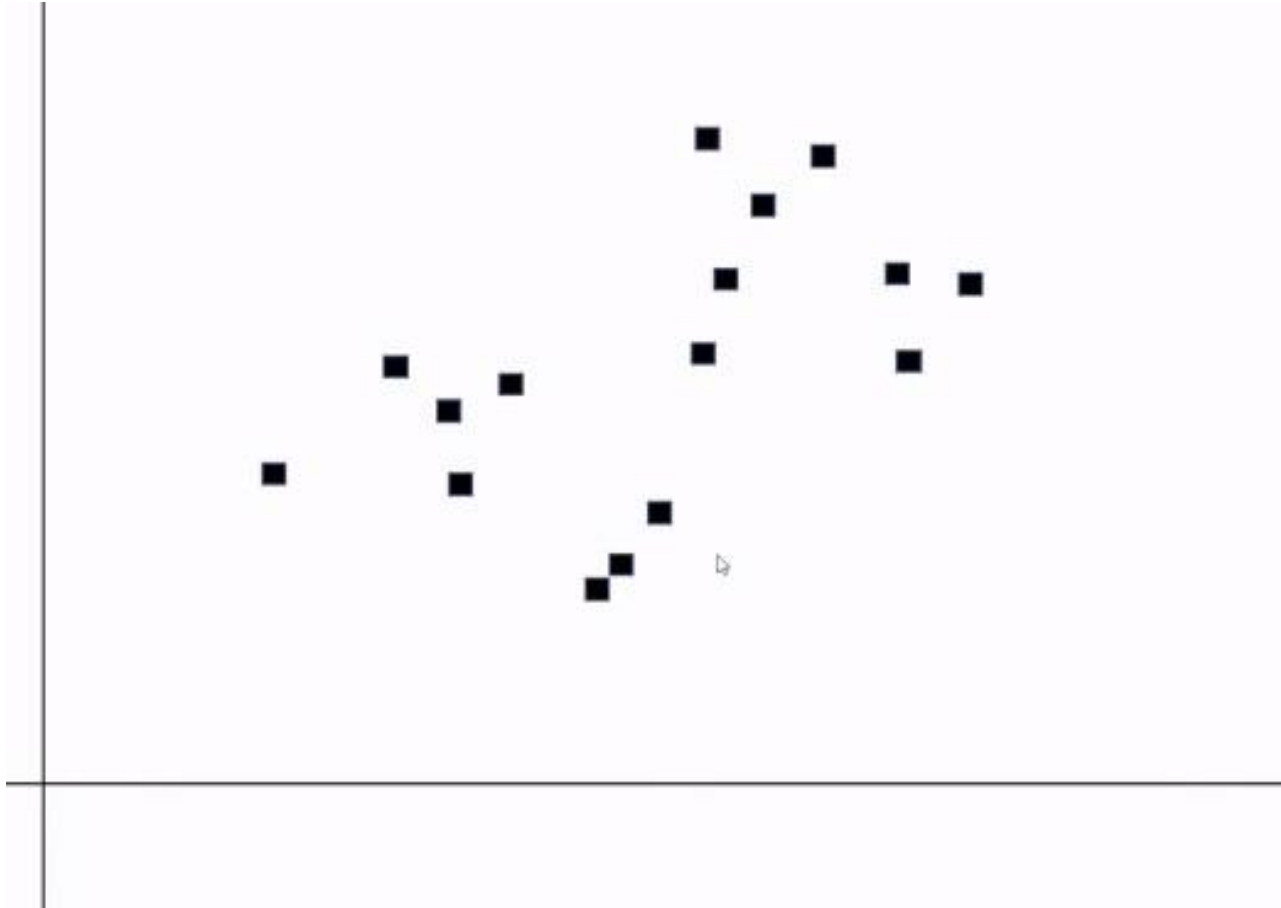


Algorithm k-means clustering

- Decide on a value for $k = \# \text{ of clusters}$: *Hyperparameter*.
- Initialize the k cluster centers/centroids (randomly based on an empirical probability distribution of the points).
- Decide the class memberships of the N objects by assigning them to the nearest cluster center.
- Re-estimate/Update the k cluster centers, by assuming the memberships found above are correct by measuring the mean position/coordinate of the data samples.
- If none of the N objects changed membership in the last iteration, exit. Otherwise go to step 3

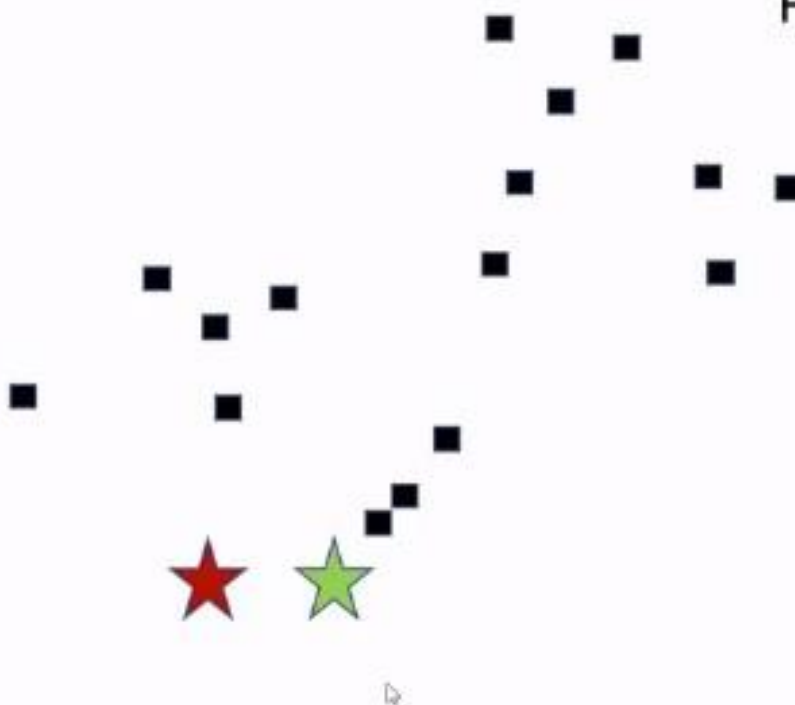


K-Means Algorithm Example

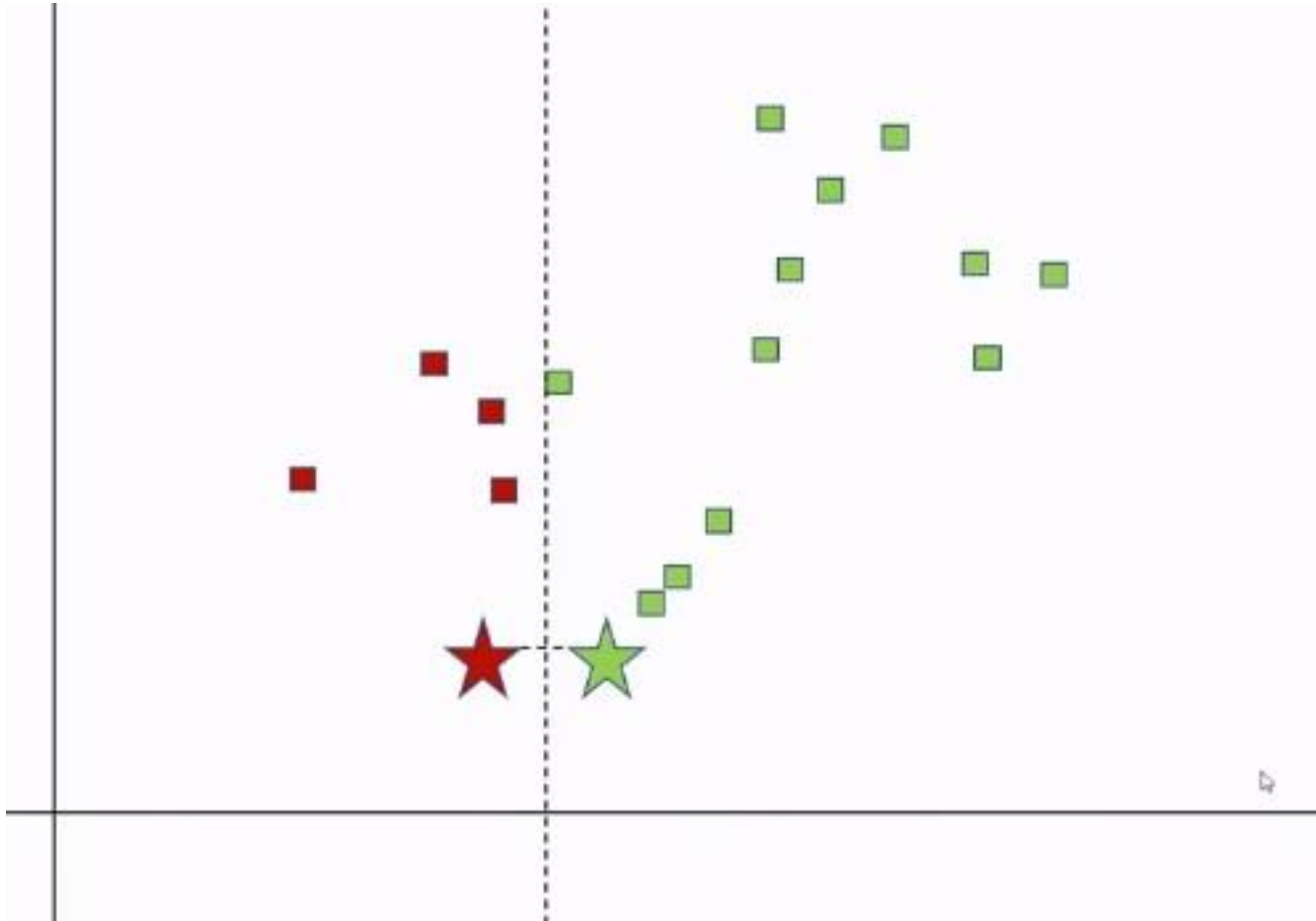


K-Means Algorithm Example

1. Start with K centroids by putting them at random place,
Here $k = 2$



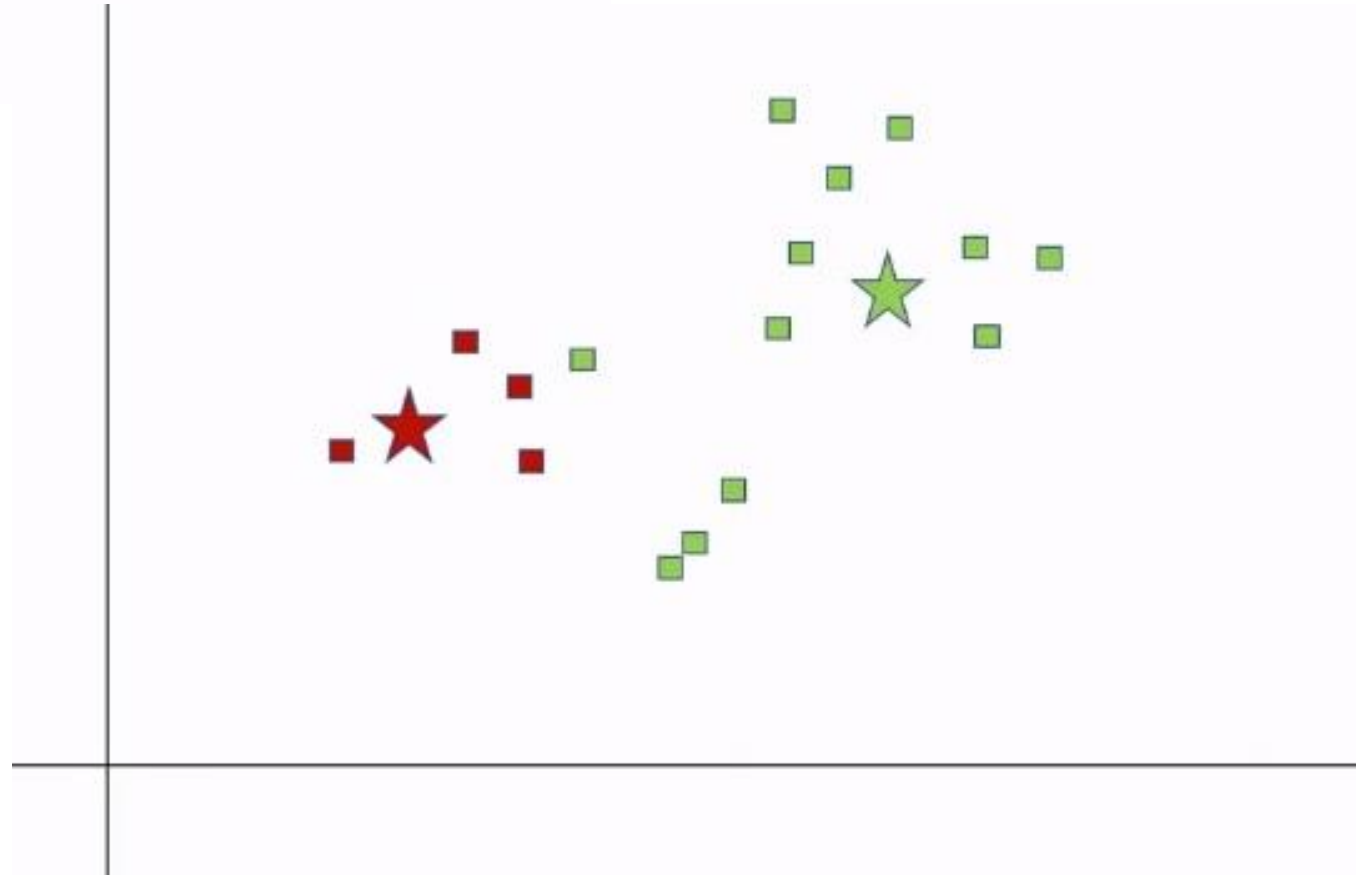
K-Means Algorithm Example



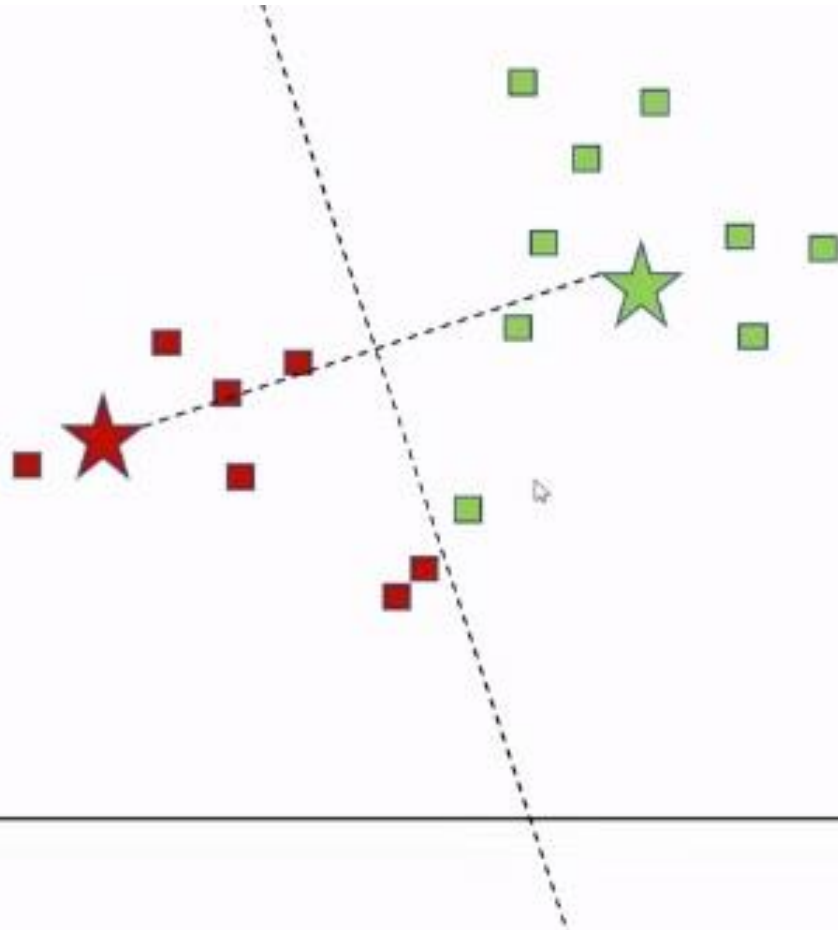
- Find a boundary
- Then label the instances, update the centroids

K-Means Algorithm Example

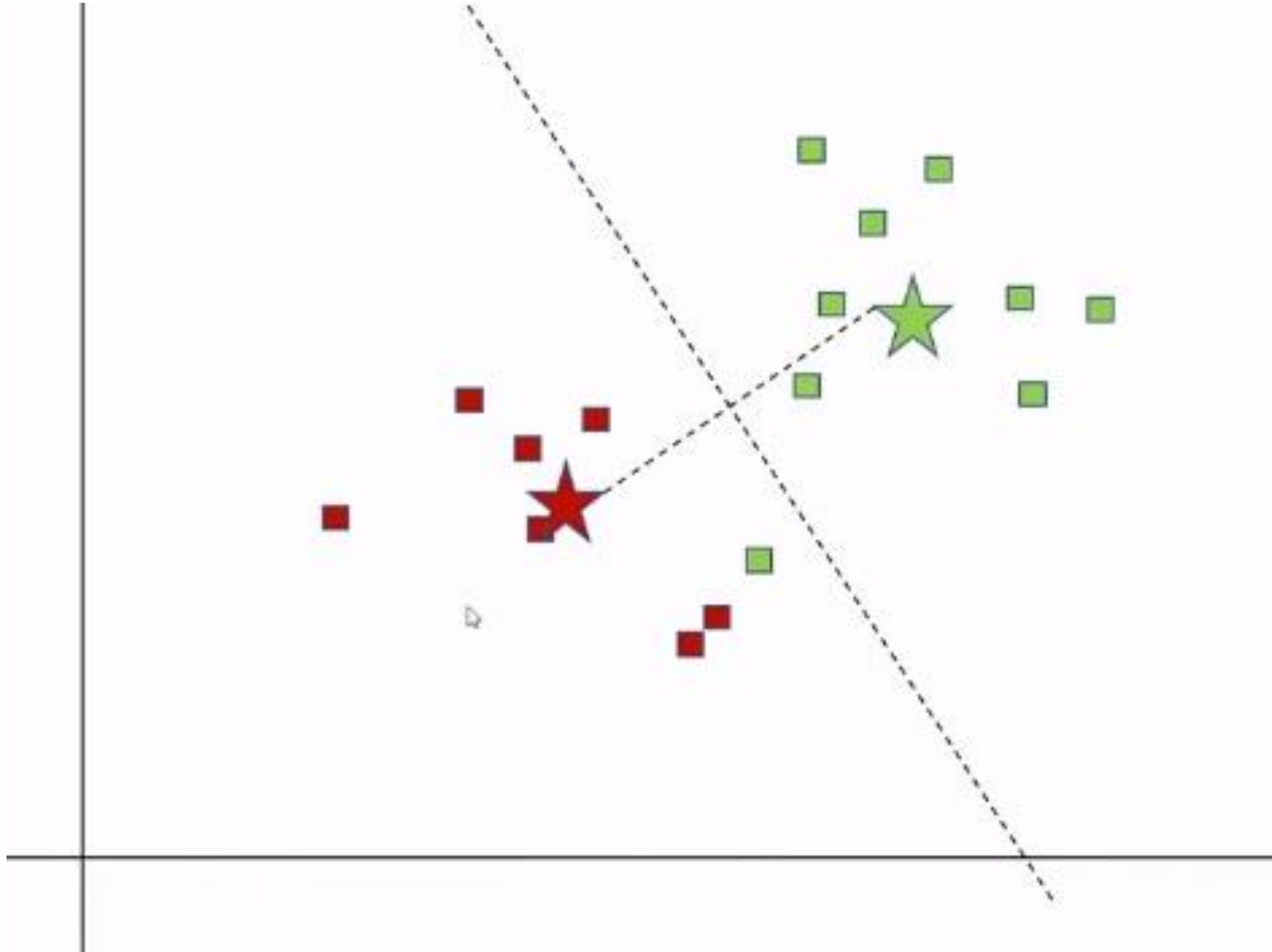
2. Compute distance of every point from centroid and cluster them accordingly

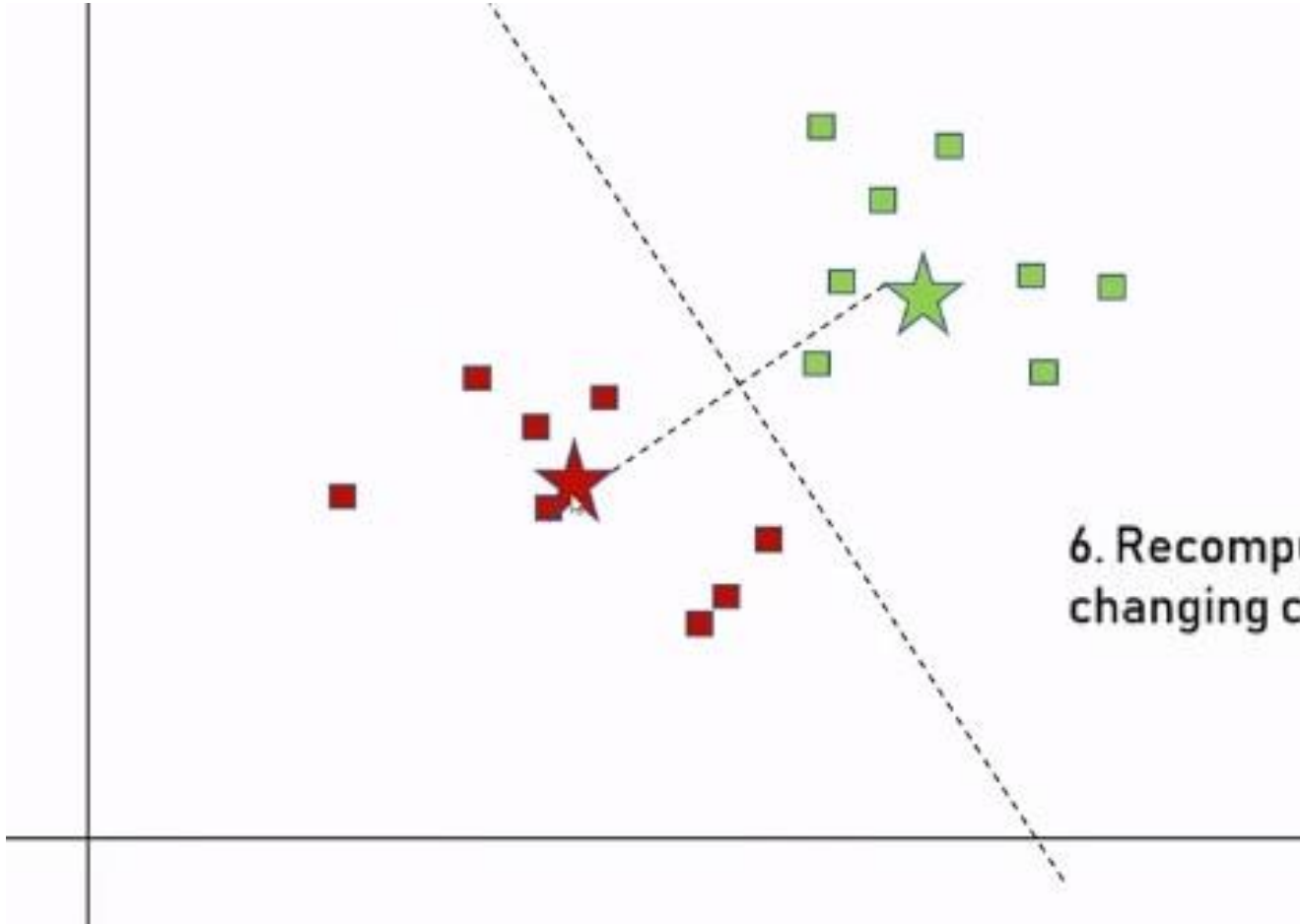


4. Again re-cluster every point based on their distance with centroid

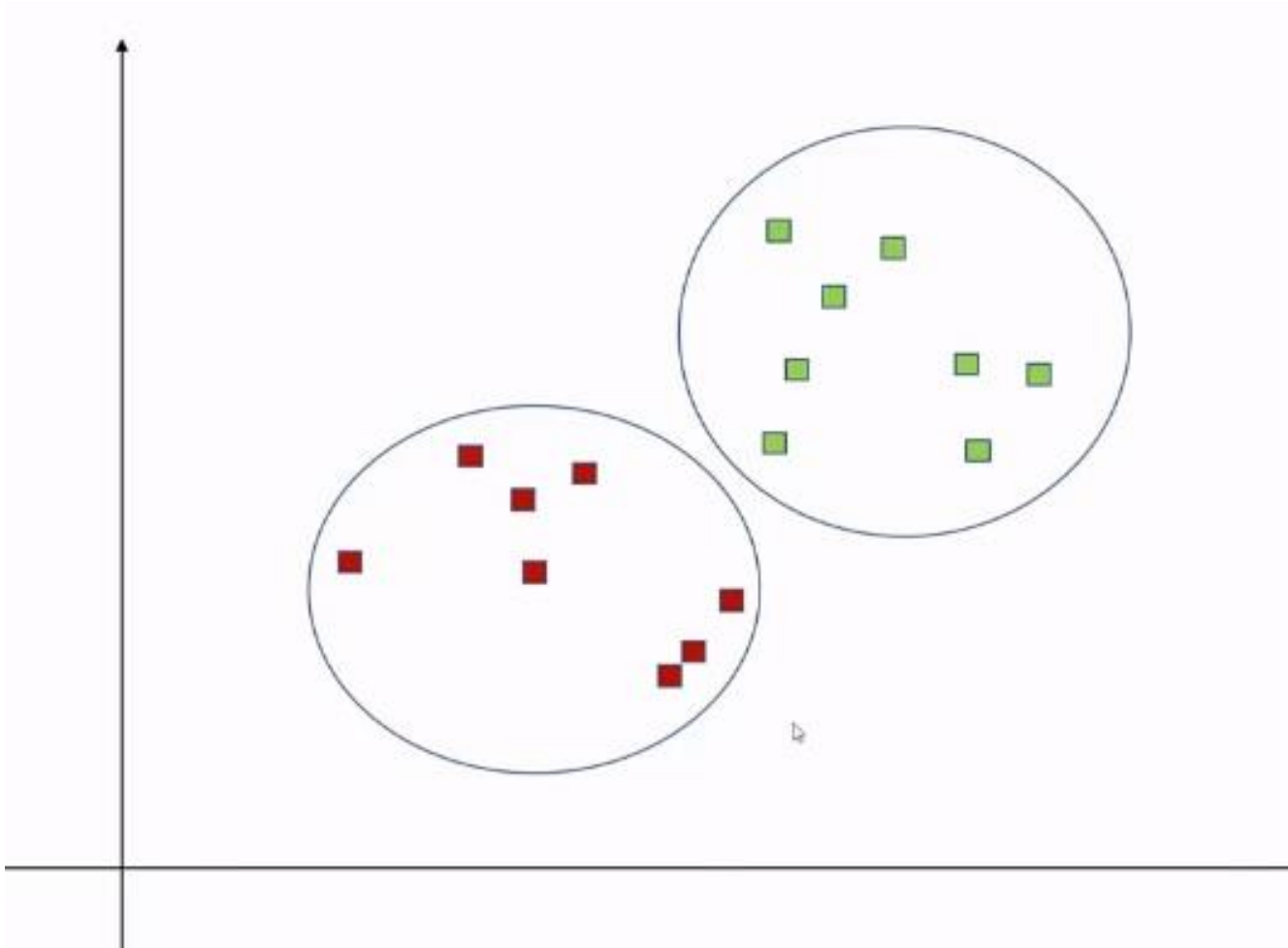


5. Again adjust centroids



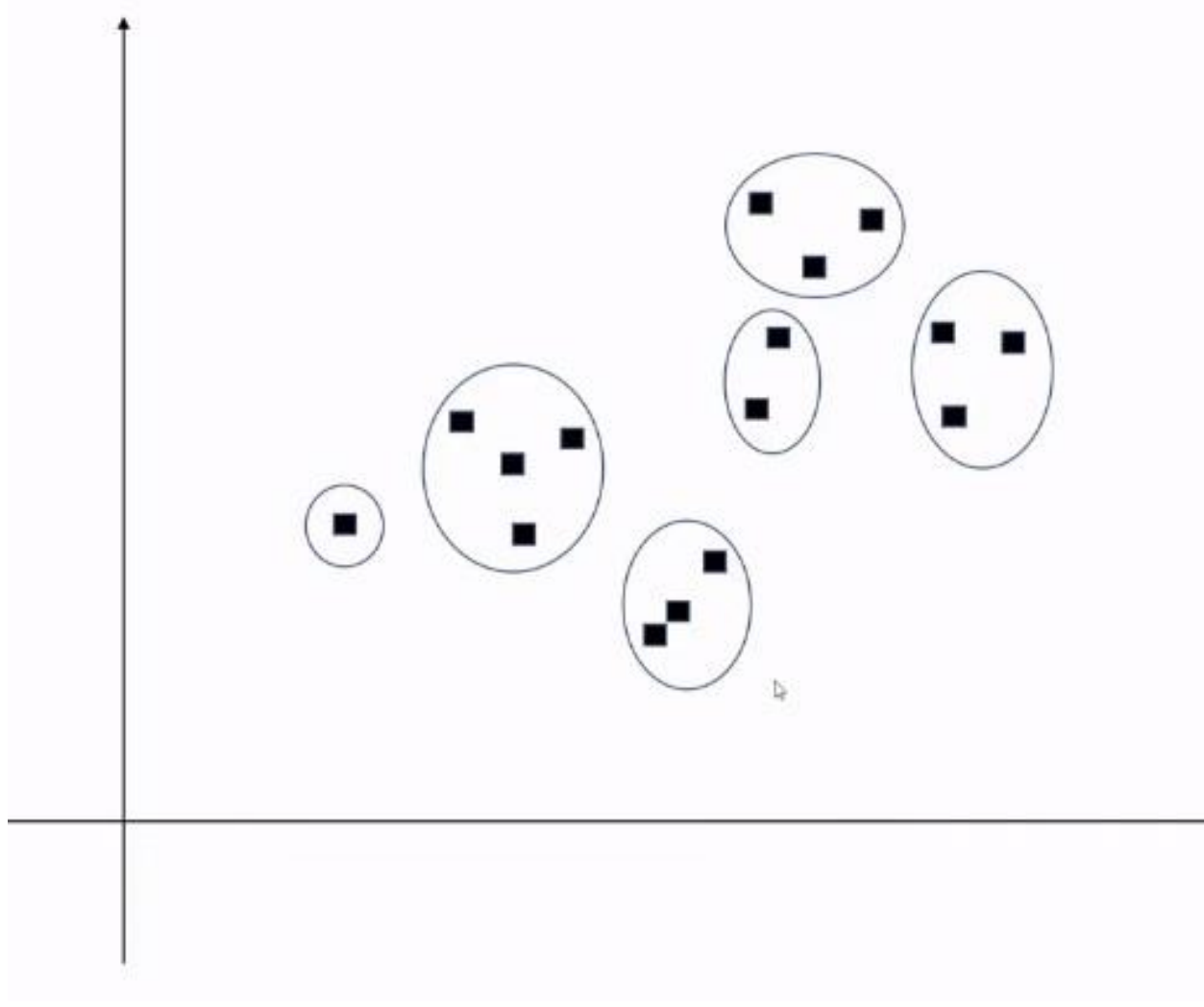


6. Recompute clusters and repeat this till data points stop changing clusters



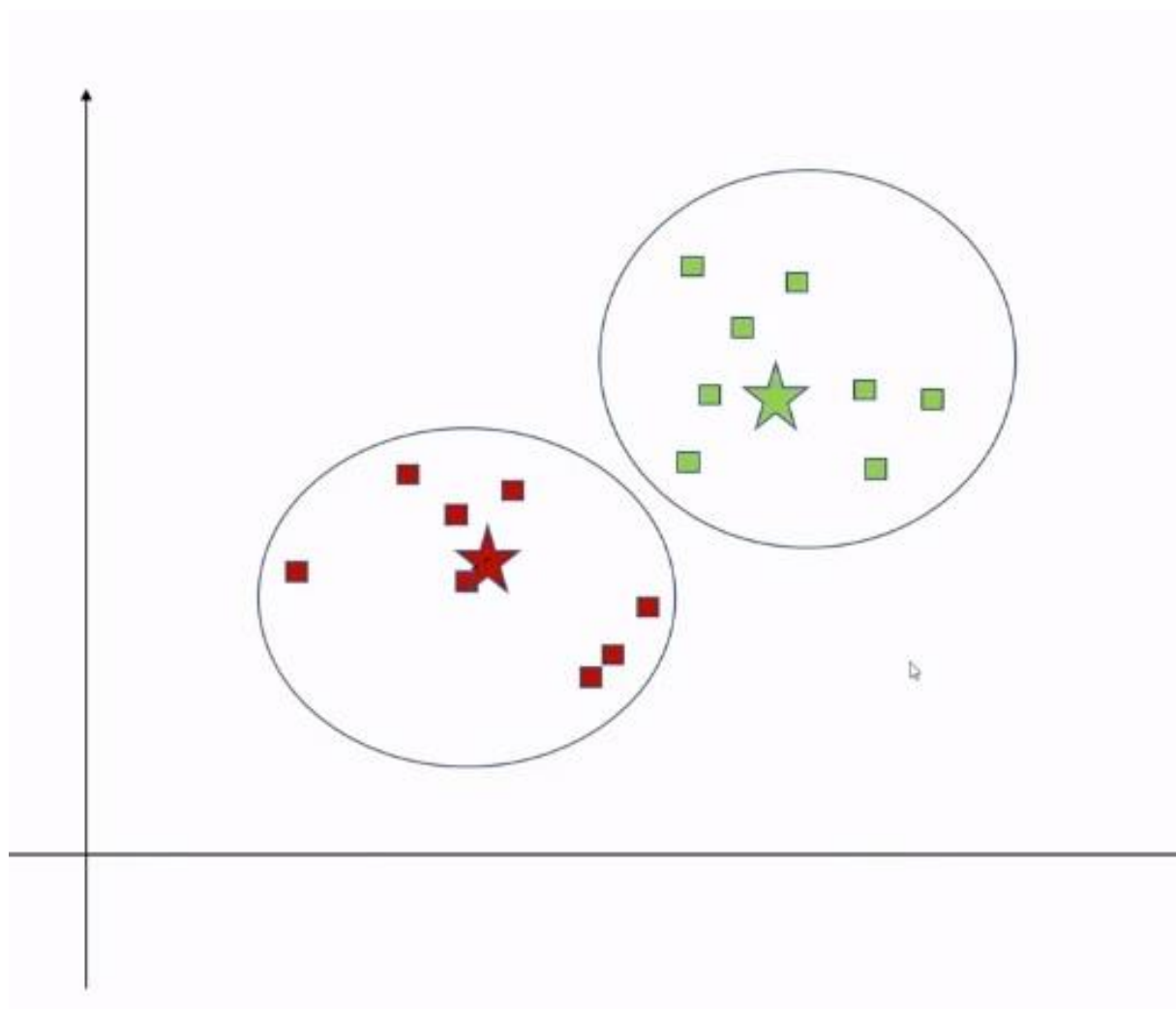
Finally got two clusters

How to find the K value?



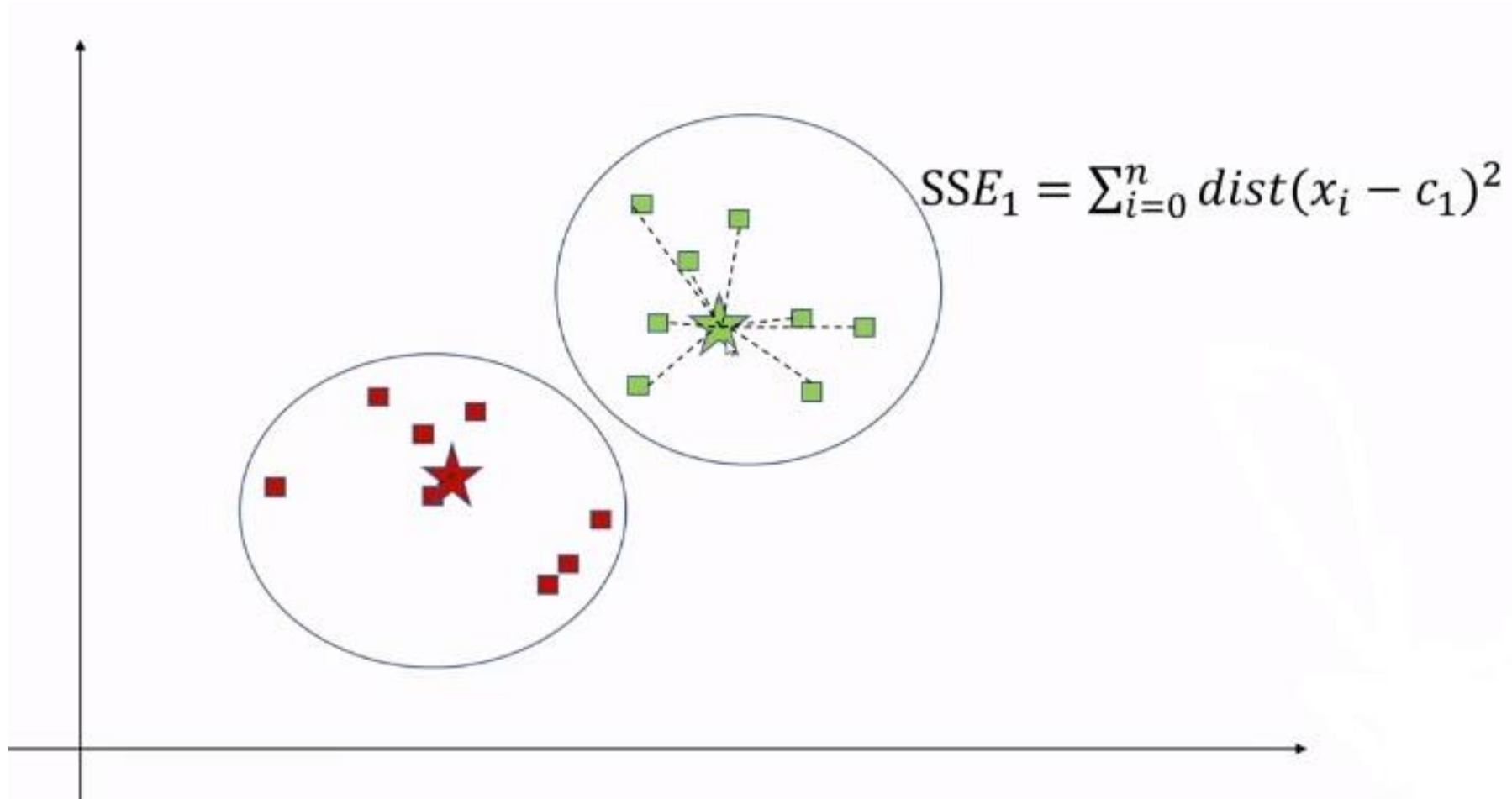
- The same dataset can be clustered into 6 clusters as well
- To decide how many cluster will be there or what will be the optimum K value, calculate Sum of Squared Errors (SSE)

How to find the K value?: SSE

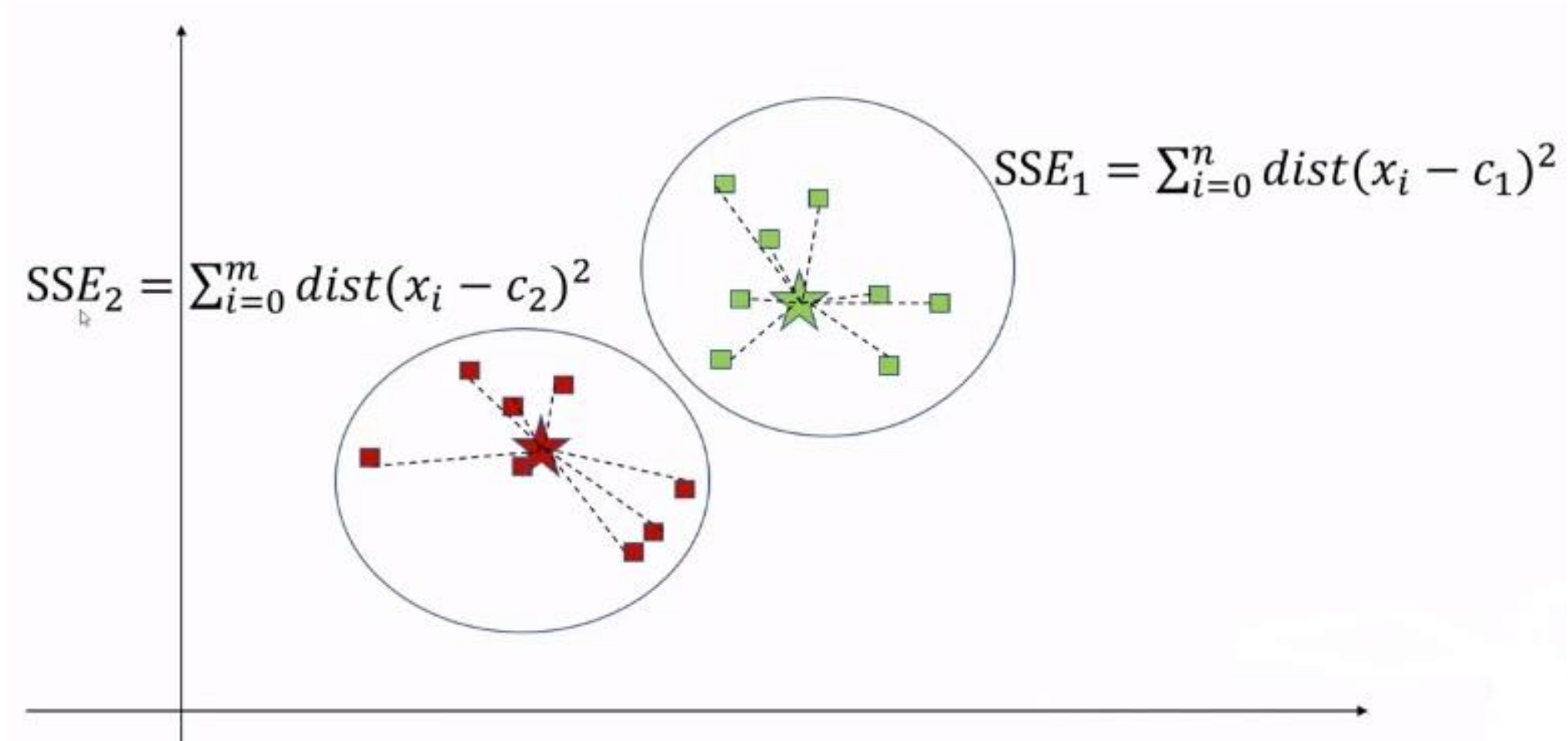


SSE = Sum of Squared Errors

How to find the K value?: SSE



How to find the K value?: SSE



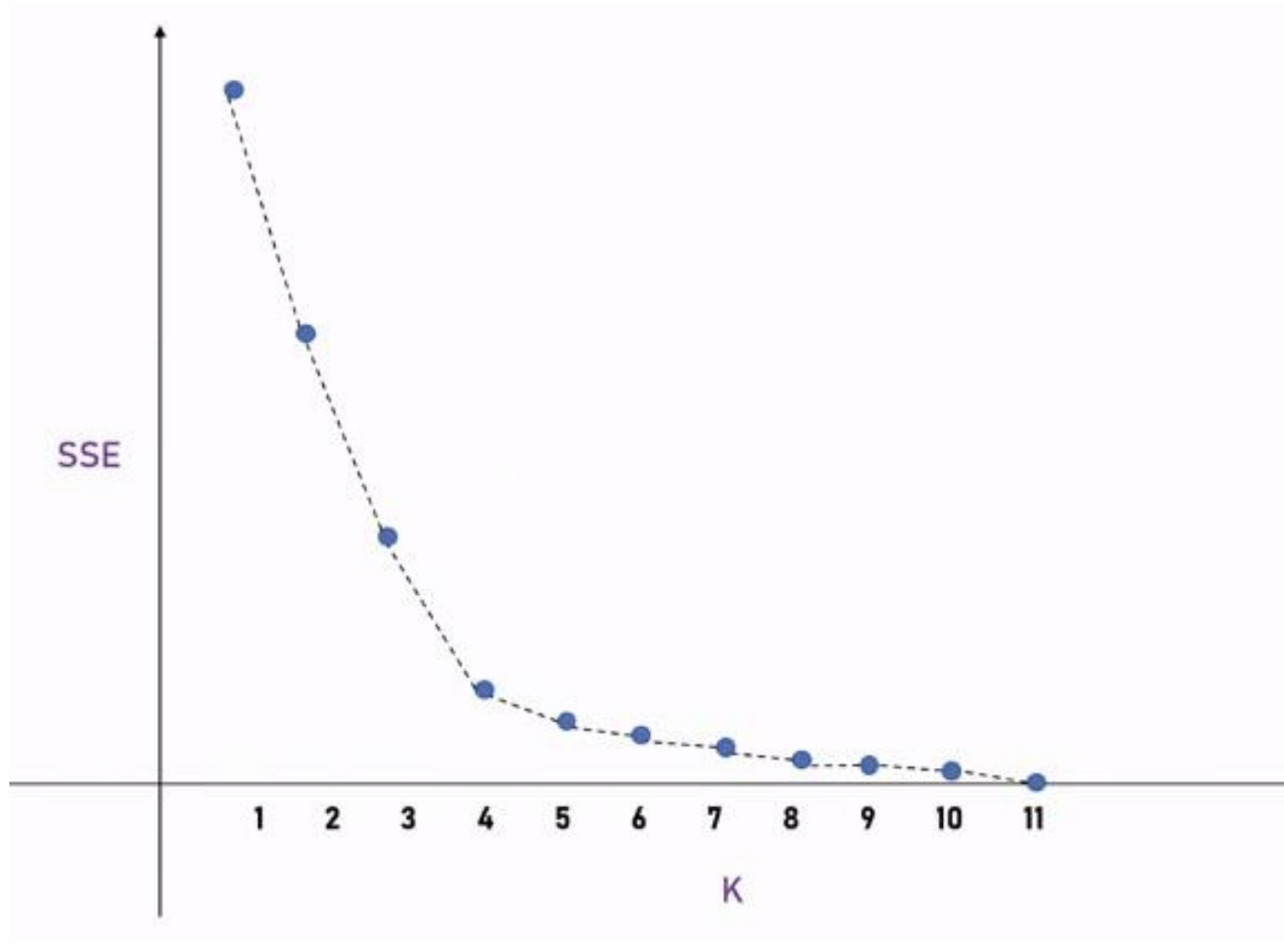
How to find the K value?: SSE

- If there are K numbers of clusters:

$$\mathbf{SSE} = \mathbf{SSE_1} + \mathbf{SSE_2} + \dots + \mathbf{SSE_k}$$

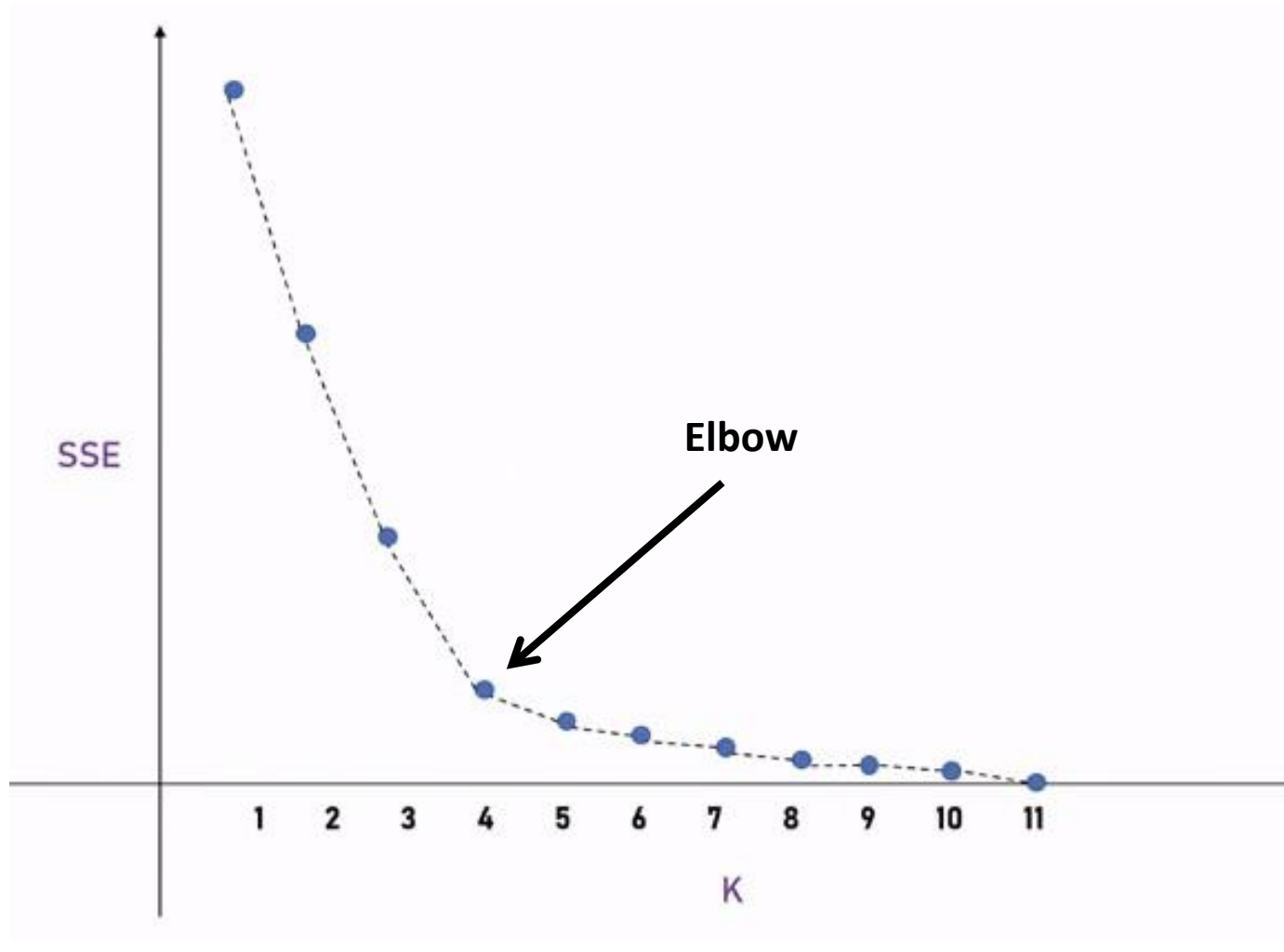


How to find the K value?: SSE



- Plot SSE for different K values
- the more clusters there are, the closer each instance will be to its closest centroid
- The more K value will be the less SSE will be

How to find the K value?: Elbow Technique



- the SSE drops very quickly as we increase k up to 4,
- but then it decreases much more slowly as we keep increasing k.
- This curve has roughly the shape of an arm
- Choose the Elbow value

sklearn for KMeans

- Let's train a K-Means clusterer on this dataset. It will try to find each blob's center and assign each instance to the closest blob

```
from sklearn.cluster import KMeans  
k = 5  
kmeans = KMeans(n_clusters=k)  
y_pred = kmeans.fit_predict(X)
```



Silhouette score

- A more precise approach is to use the *silhouette score*, which is the mean *silhouette coefficient* over all the instances.
- but this process is more computationally expensive
- Silhouette coefficient = $(b - a) / \max(a, b)$
 - Here, a is the mean intra-cluster distance, the **mean distance to the other instances** in the **same cluster**
 - b is the mean nearest-cluster distance, the mean distance to the instances of the next closest cluster



Silhouette score

- The silhouette coefficient can vary between -1 and $+1$
- A coefficient close to $+1$ means that the instance is well inside its own cluster and far from other clusters
- a coefficient close to 0 means that it is close to a cluster boundary
- a coefficient close to -1 means that the instance may have been assigned to the wrong cluster



Silhouette score

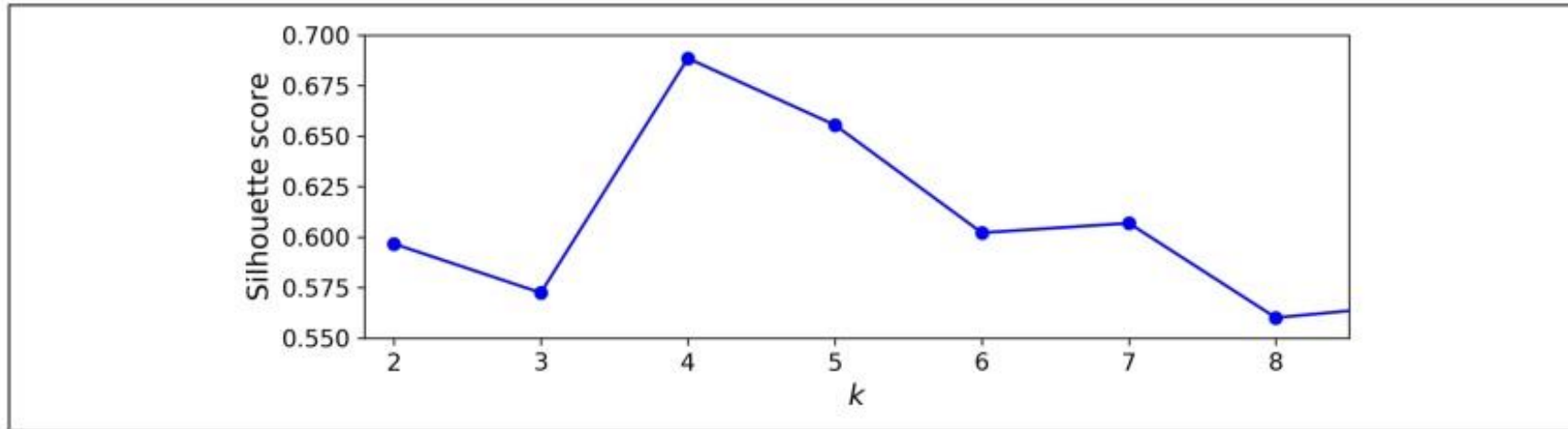
- To compute the silhouette score, we can use Scikit-Learn's `silhouette_score()` function

```
>>> from sklearn.metrics import silhouette_score  
>>> silhouette_score(X, kmeans.labels_)
```

0.655517642572828



Silhouette score



- *Figure is showing how to select the number of clusters k using the silhouette score*
- It confirms that $k = 4$ is a very good choice
- it also underlines the fact that $k = 5$ is quite good as well, and much better than $k = 6$ or 7.
- This was not visible when comparing inertias.

Strength and Weakness

- Strength
 - *Relatively efficient: $O(tkn)$* , where n is # objects, k is # clusters, and t is # iterations. Normally, $k, t \ll n$.
 - Often terminates at a *local optimum*.
- Weakness
 - Applicable only when *mean* is defined, then what about categorical data?
 - Need to specify k , the *number* of clusters, in advance
 - Unable to handle noisy data and *outliers*



Clustering Applications

- Clustering is used in a wide variety of applications:
- *For customer segmentation*
 - we can cluster your customers based on their purchases and their activity on our website. This is useful to understand who our customers are and what they need, so we can adapt your products and marketing campaigns to each segment.
- *For data analysis*
 - When you analyze a new dataset, it can be helpful to run a clustering algorithm, and then analyze each cluster separately



Clustering Applications

- *As a dimensionality reduction technique*
 - Once a dataset has been clustered, it is usually possible to measure each instance's *affinity with each cluster*. Each instance's feature vector **x** **can then be replaced with the vector of** its cluster affinities. If there are *k clusters*, *then this vector is k-dimensional*. This vector is typically much lower-dimensional than the original feature vector
- *For anomaly detection (outlier detection)*
 - Any instance that has a low affinity to all the clusters is likely to be an anomaly. Anomaly detection is particularly useful in detecting defects in manufacturing, or for *fraud detection*.



Clustering Applications

- *For semi-supervised learning*
 - If you only have a few labels, you could perform clustering and propagate the labels to all the instances in the same cluster
- *For search engines*
- *To segment an image*
 - By clustering pixels according to their color, then replacing each pixel's color with the mean color of its cluster, it is possible to considerably reduce the number of different colors in the image

